CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Experiment 05

Name	: Premdeep Singh	**UID**	: 23BCS11890
Branch	: CSE	**Section**	: KRG-3A
Semester	: 6	**Date of exp.**	: 26/02/26
Subject	: System Design	**Subject Code**	: 23CSH-314

AIM

To design a real time chatting application like WhatsApp/Messenger that supports one-to-one and group messaging with high availability, low latency, and reliable message delivery.

OBJECTIVE:

- Understand the architecture of a real-time messaging system.
- Identify functional requirements such as authentication, chat, and media sharing.
- Identify non-functional requirements including availability, reliability, and latency constraints.
- Analyze CAP theorem trade-offs for messaging systems.
- Design REST and WebSocket APIs for chat communication.

REQUIREMENTS

Functional Requirements:

1. Client should be able to register and login into the system.
2. System should support one-to-one messaging.
3. System should support group messaging.
4. System should support sending textual as well as media messages.
5. System should preserve complete message history.
6. Client should be able to see delivered and read receipts.

7. Messages should be received in real-time.

Non-Functional requirements:

- System should handle extremely large scale traffic (millions to billions of messages per day) requiring approximately 100 TB storage. ○ High Availability should be prioritized under CAP theorem.

- System should follow Eventual Consistency model.
- End-to-end latency should be approximately 200–300 ms. ○ System should be highly reliable with no message loss or packet drop.
- Horizontally scalable distributed architecture

HIGH LEVEL DESIGN

Core-Entities

- ✦ Users
- ✦ Groups
- ✦ Messages

Attached ott.drawio file has whole high-level design.

LOW LEVEL DESIGN

API-Endpoints

1. User Registration API:
   POST /user/register
2. User Login API:
   POST /user/login
3. One-to-One Messaging
   A. Send Message (WebSocket): WS /messages/send
   B. Get Chat List: GET /chat/{user_id} (Pagination Supported)

C. Get Messages of Specific Chat:
GET /messages/{user_id}/{receiver_id}

4. Group Messaging
   A. Create Group: POST /groups/create
   B. Send Group Message (WebSocket): WS /messages/send
   C. Get Group Chats: GET /groups/{group_id} (Pagination Supported)
   D. Add Member: POST /groups/{group_id}/add
   E. Remove Member: DELETE /groups/{group_id}/remove

LEARNING OUTCOMES

- Designed a scalable real-time messaging system.
- Identified functional and non-functional requirements. o Designed REST and WebSocket APIs.

- Understood high availability and eventual consistency trade-offs in messaging systems.